

Technical Learnings and Experiments: Logic & SAT Solving

1 Boolean Satisfiability and CNF

The core of this project involved translating high-level problem statements into Boolean logic. We utilized **Boolean Satisfiability (SAT)**, which determines if there is an interpretation (assignment of True/False to variables) that satisfies a given Boolean formula.

Conjunctive Normal Form (CNF)

We learned that SAT solvers require input in CNF. This structure consists of a conjunction (AND) of clauses, where each clause is a disjunction (OR) of literals.

- **Structure:** $(A \vee B) \wedge (\neg B \vee C \vee \neg D) \wedge \dots$
- **Conversion:** We practiced converting logical implications. For example, $A \implies B$ is converted into its CNF equivalent $\neg A \vee B$.

2 Encoding Constraints in Sudoku

Building the Sudoku solver provided a foundation for static constraint modeling.

Variable Mapping

We mapped the 3D coordinate system $(row, column, digit)$ into a single integer ID to interface with the SAT solver. The formula used was:

$$(row \times 9 + col) \times 9 + digit \tag{1}$$

Constraint Definitions

- **“At Least One” Constraint:** To ensure a cell (r, c) is not empty, we used a clause containing all possible digits:

$$(X_{r,c,1} \vee X_{r,c,2} \vee \dots \vee X_{r,c,9})$$

- **“At Most One” Constraint:** To prevent multiple numbers in a single cell, we generated pairwise negative clauses. For any two digits d_1 and d_2 in the same cell:

$$(\neg X_{r,c,d_1} \vee \neg X_{r,c,d_2})$$

This concept was critical and reused later in the Sokoban project to prevent multiple objects from occupying the same space.

3 Dynamic Systems in Sokoban

The Sokoban project introduced the dimension of **Time**, requiring a transition-based model (Bounded Model Checking).

3.1 Variable Expansion

We expanded our variable state to include time t . This allowed the solver to distinguish between “Player is at (0,0) at start” and “Player is at (0,0) after 1 move.”

- $Player(row, col, t)$
- $Box(row, col, t)$
- $Action(t)$

3.2 Frame Axioms

A major technical challenge was the **Frame Problem**. In logic, variables do not retain their value automatically between time steps. If we did not explicitly constrain a box to stay put, the solver would toggle it to False to satisfy other conditions easily.

Implementation: We implemented explicit “Inertia” constraints:

$$Box(r, c, t + 1) \iff (Box(r, c, t) \wedge \neg PushedAway) \vee PushedHere \quad (2)$$

In CNF, this required defining both directions:

1. If a box is at (r, c) at t and no action pushes it away, it must be at (r, c) at $t + 1$.
2. If a box is at (r, c) at $t + 1$, it must have either been there at t or pushed there.

3.3 Preventing Teleportation

We discovered that defining valid moves (e.g., “Player can move Right”) was insufficient. The solver would “teleport” the player to the necessary starting position for a push.

Fix: We enforced strict arrival constraints. A player variable $P_{r,c,t+1}$ can only be True if one of the valid “causes” occurred at time t (e.g., player was at neighbor $P_{r-1,c,t}$ and Action was ‘Down’).

4 Combinatorial Implementation

This was necessary to distribute the “OR” logic over multiple conditions when generating CNF clauses for the Frame Axioms.

- **Logic 1:** To encode $A \implies (B \vee C)$, we convert to $\neg A \vee B \vee C$.
- **Logic 2:** To encode $Y \implies (A \wedge B)$, we generate two clauses: $(\neg Y \vee A)$ and $(\neg Y \vee B)$.
- **Logic 3:** To encode $Z \implies ((A \wedge B) \vee (C \wedge D))$, which represents multiple possible “causes” for a state (like a box arriving), we required distributing the OR over the ANDs. This resulted in multiple clauses representing every combination of literals.